

コンピュータアーキテクチャI

担当：佐々木 敬泰

第4回

様々なプロセッサと命令セット

今日の内容

- プロセッサの分類
- 命令の基本
- 命令セット
- レジスタセット
- アドレッシングモード
- RISCとCISC

コンピュータアーキテクチャI

3

シラバス

1. イントロダクション
2. コンピュータの構成
3. 数の表現
4. 様々なプロセッサと命令セット
5. 基本命令
6. メモリアクセス命令
7. 分岐命令
8. 高級言語からアセンブラへの変換
9. プログラムの翻訳と起動
10. 加減算
11. 乗算アルゴリズムと乗算器
12. 除算アルゴリズムと除算器
13. 浮動小数点
14. 浮動小数点の乗算と誤差
15. 定期試験

コンピュータアーキテクチャI

2

プロセッサの分類

コンピュータアーキテクチャI

4

プロセッサの種類

- 普段使ってるパソコンやスマホのCPUがどんなものか知ってる人？
 - PC
 - Intel Core i3/i5/i7/i9
 - Intel Xeon
 - AMD Ryzen Threadripper
 - Apple M1
 - スマホ
 - ARM系
いろいろなメーカーが設計・製造してる
Qualcomm, MediaTek, Apple, Intel, ...

コンピュータアーキテクチャI

5

Intelプロセッサの例（抜粋）

- 8086
- 80286
- 80386
- 80486
- 80586(Pentium)
- Core2
- Core i3/i5/i7
 - Nehalem
 - Sandy Bridge
 - Ivy Bridge
 - ...
 - Coffee Lake

16bit CPU
(x86)

32bit CPU
x86

64bit CPU
x64

機能や性能で上位に位置づけられる製品が、下位の製品と**互換性**をもつこと

基本的には**上位互換**あり（一部例外あり）

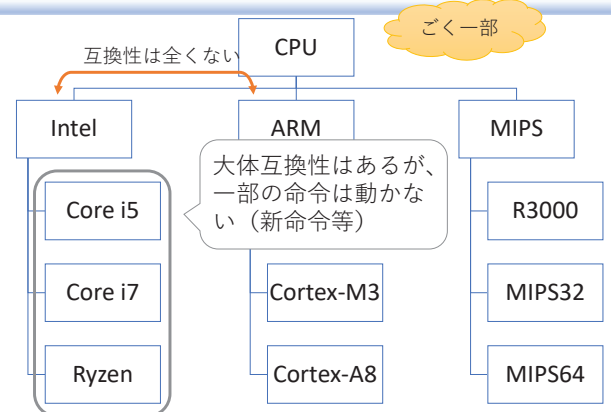
ここ重要！



コンピュータアーキテクチャI

7

プロセッサの分類

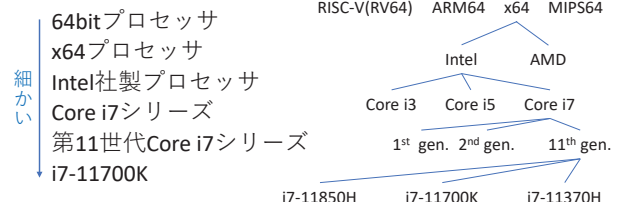


コンピュータアーキテクチャI

6

ブランド名？シリーズ名？製品名？

- プロセッサの分類方法はいろいろ
- Intel Core i7-11700K Processor
 - Intel社のプロセッサ
 - 開発コード：Rocket Lake
 - 製品分類：



コンピュータアーキテクチャI

8

OSの分類

• 主なOSの種類

- Windows
 - x86→x64(x86), ARM
- Linux系
 - x86, x64, ARM, MIPS, RISC-V, ...
- MAC OS
 - 68000→PowerPC→Intel Core→Apple M1(ARM)
- Android
 - ARM, (MIPS, x86, ...)
- iOS
 - ARM

【発展】
仮想化、エミュレーション

OSやプロセッサが違えば
プログラムは動かない

命令の基本

プロセッサは単純なことしかできない

• コンピュータのできること

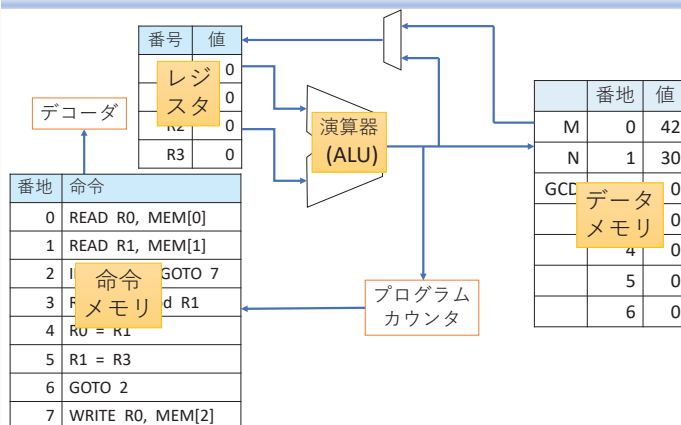
- **事前に**与えられた指示に従って**順番に**動くこと

• 単純な作業しかできない

- 2つの数を足す、あるいは引く
- 2つの数を掛ける、などなど (2項演算)
- 2つの数を比較する
- ある条件を満たしたときに、プログラムの特定の行に飛ぶ
- メモリ (メモ帳) から数を読み出す
- メモリ (メモ帳) に数を書き出す

➡ 基本的にこれしかできない

プロセッサの構造



アセンブラと機械語

• プロセッサが解釈できるのは機械語のみ

- 人間には読みにくいので、機械語と (ほぼ) 一対一対応したアセンブラを用いる

【アセンブラ】	【機械語 (16進数表記)】
push %rbp	55
mov %rsp, %rbp	48 89 e5
sub \$0x10, %rsp	48 83 ec 10
movl \$0x55, -0x10(%rbp)	e7 45 f0 55 00 00 00
movl \$0x2d, -0xc(%rbp)	45 f4 2d 00 00 00 00
movl \$0x5a, -0x8(%rbp)	45 f8 5a 00 00 00 00
movl \$0x0, -0x4(%rbp)	45 fc 00 00 00 00 00
jmp L1	eb 1f
mov -0x4(%rbp), %eax	8b 45 fc

それぞれ
対応する

MIPSの機械語

• 1命令32bitの固定長

• MIPSの命令フォーマット

• Type I

- 即値 (定数) を取る書式

命令	Reg.	Reg.	即値 (定数)
----	------	------	---------

• Type J

- 分岐 (ジャンプ) 命令用の特殊フォーマット

命令	飛び先アドレス
----	---------

• Type R

- オペランドとしてレジスタのみを取る

命令	Reg.	Reg.	Reg.	shamt	機能
----	------	------	------	-------	----

アセンブラの基本的な書き方(1/2)

• 1行に1命令

命令 オペランド, オペランド, ...

レジスタや即値
(定数)、ラベル等

• レジスタは\$ + レジスタ番号で表記

- 定数部は10進数(123)、16進数(0x32)、8進数(017)で表す

• 例)

- ADD \$1, \$3, \$6
- ADDI \$2, \$1, 13

アセンブラの基本的な書き方(2/2)

• ラベル (後述の分岐命令等で利用)

- ラベル名 + :

- 例)

L1: ADDI \$1, \$2, 1
J L1

• コメント

- #以降は行末まで無視される

ADDI \$1, \$1, 1 # Increment loop counter

アセンブラは処理系に強く依存する。
ターゲットCPUが違えば互換性はほぼ
ない。Intel系は処理系によってAT&T構
文、Intel構文があって互換性もない。

命令セット

命令セット

- 命令セット
 - プロセッサで実行できる命令の集合
 - x86命令セット
 - Intel 386、Pentium、XEON等で実行できる
 - AMD AthlonやRyzen等でも実行できる
 - ARM命令セット
 - ARM Cortex-AやCortex-M等で実行できる
- 互換性
 - 一般的には命令セット間に互換性はない
 - ⇒Intel CPU上でARMのプログラムは動かない
 - 例外) x64はx86を含む

命令の分類

- 機能による分類
 - 算術演算
 - 加減乗除、剰余等
 - 論理演算
 - ビット和・積・排他的論理和・シフト等
 - 定数ロード命令
 - メモリアクセス命令
 - メモリから読み出し、メモリへの書き込み
 - 分岐命令
 - 条件分岐
 - 無条件分岐
 - 疑似命令
 - 1つ以上の命令の組み合わせで実現する

レジスタ(1/2)

- 一般にメモリは（CPUの動作速度に比べて）遅い
 - CPU：1～3GHz
 - メモリ(DRAM)：400～800MHz
- 命令実行の度にデータメモリにアクセスするのは非効率
 - CPU内に小型のメモリ（レジスタ）を用意し、**レジスタ間でのみ演算**を行う
 - 必要に応じて、メモリとレジスタ間でデータを交換する

プロセッサと機械語

- 命令はプロセッサ毎に違う
 - Intel X86系
 - Intel X64系（X86系の命令も実行可）
 - ARM系
 - MIPS系
 - RISC-V系
 - SH系
 - PIC系
 - AVR系
 - 他いろいろ

命令セット・アーキテクチャ(ISA)

- 命令セット・アーキテクチャ
 - Instruction Set Architecture (ISA)
 - 機械語の**仕様**を定めたもの
 - 命令の種類や動作
 - 機械語のフォーマット
 - レジスタ数
 - 実際のプロセッサアーキテクチャ（マイクロアーキテクチャ）とは異なる
 - ソフトウェアからみた（仮想的な）コンピュータの仕様

レジスタ

プロセッサ内にある値を一時的に記憶する箱

レジスタ(2/2)

- レジスタはプロセッサによって異なる
 - Intel 8086
 - 16bitレジスタ8本
 - Intel 80386
 - 32bitレジスタ8本
 - MIPS
 - 32bitレジスタ32本
 - ARM
 - 32bit/64bitレジスタ16～32本
 - Intel x86 (Core i5/i7/i9等)
 - 64bitレジスタ16本
- ➡ 上記に加え、制御用レジスタを複数本持っている

MIPSのレジスタ構成

汎用レジスタ

レジスタ番号	機能
\$0	ゼロレジスタ
\$1	汎用レジスタ
\$2	汎用レジスタ
\$3	汎用レジスタ
⋮	⋮
\$31	汎用レジスタ

JAL命令は例外
ハードウェア上の区別はない

- MIPSでは、レジスタを\$+数字で表す
- \$0は常に0（何を書き込んでも無視）
⇒非常に重要（今後も出てくる）

25

MIPSのレジスタ割り当て

レジスタの用途

名前	レジスタ番号	用途	呼び出し時に退避
\$zero	0	常にゼロ	不要
\$v0 - \$v1	2 - 3	結果、式の評価	不要
\$a0 - \$a3	4 - 7	関数の引数	不要
\$t0 - \$t7	8 - 15	一時的な変数	不要
\$s0 - \$s7	16 - 23	退避	必要
\$t8 - \$t9	24 - 25	予備の一時変数	不要
\$gp	28	グローバルポインタ	必要
\$sp	29	スタックポインタ	必要
\$fp	30	フレームポインタ	必要
\$ra	31	戻りアドレス	必要

\$26 - \$27はオペレーティングシステム用に予約されている

27

MIPSのアドレッシングモード

- アドレッシングモードとは？
 - オペランド（操作対象）やアドレスを表現する方法
- MIPSのアドレッシングモード
 - 即値アドレッシング
 - 即値（定数）
 - レジスタアドレッシング
 - レジスタ値
 - ベース相対アドレッシング
 - レジスタ値 + 即値
 - PC相対アドレッシング
 - PC + 即値
 - 疑似直接アドレッシング
 - PCの一部と即値の一部を結合

コンピュータアーキテクチャI

29

RISCとCISC

- RISC
 - Reduced Instruction Set Computer
 - 少ない命令の組み合わせからなる命令セットアーキテクチャ (ISA : Instruction Set Architecture)
 - CISC
 - Complex Instruction Set Computer
 - 複雑な命令を含むISA
- どちらも命令セットの話で、実際のマイクロプロセッサアーキテクチャとは別

31

レジスタの使い方

- ハードウェア的には、\$1～\$31は区別がない（けれど）、
 - レジスタの使い方を決めないとソフトウェアの互換性が保てない
 - 例)
 - Aさんの作った関数は引数を\$2, \$3に入れる
 - Bさんの作った関数は引数を\$10, \$11に入れる
- ⇒こうならないように規則ABI(Application Binary Interface)が定められてる
- 既存のライブラリやOS、コンパイラ等はこれに従っている
 - アセンブラを手で書く場合も従う必要がある（OS含むすべてのコードを自分で書くなら不要）

26

アドレッシングモード

コンピュータアーキテクチャI

28

RISCとCISC

コンピュータアーキテクチャI

30

CISC命令

- CISCとは？
 - 複雑な命令
 - 命令及びアドレッシングモードが豊富
 - 具体的には？？
- 具体例
 - Push/Pop
 - 連続データ転送
 - など...

32

Push/Pop

- 一時的にデータを退避/復帰することは多い
- 通常、スタック（と呼ばれるデータ構造）を用いて実現する
 - スタック＝データを積める棚
 - 取り出すのは積んだ順
 - 例)
 - push A
 - push B
 - pop (Bが取り出される)
 - pop (Aが取り出される)



33

Push/Pop

- RISC命令
 - push \$4
sw \$4, 0(\$SP)
addi \$SP, \$SP, -1
 - pop \$4
addi \$SP, \$SP, 1
lw \$4, 0(\$SP)
それぞれ2命令に分解される
- CISC命令
 - push/popが1命令（≠1サイクル）で実行できる

34

CISCっぽい命令

- rep stos dword ptr [%edi]
 - 以下の処理を1命令で行う
- ```
while(%ecx != 0) {
 メモリ[%edi] = %eax
 %edi = %edi + 1
 %ecx = %ecx - 1
}
```
- 非常に複雑で実行には複数サイクル必要

35

## CISC命令をRISC命令で実装

- MIPS命令でrep stos命令を実装

```
$6 = %eax
$5 = %ecx
rep stos dword ptr $4
 beq $5, $0, L2
L1:
 addiu $5, $5, -1
 sw $6, 0($4)
 addiu $4, $4, 4
 bne $5, $0, L1
L2:
```

命令数 ≠ 実行速度  
に注意！

CISCで1命令だった  
ものがRISCでは5命令  
に分解される

36